A Formalized Geometric Decomposition System for Chanlun

From Fractals to Unique Segment Decomposition

Klaus

April 2026

Abstract

We present a fully formalized geometric decomposition system inspired by Chanlun technical analysis. We define fractals, strokes, and segments over finite normalized interval sequences, with all predicates explicitly operationalized and all conventions parameterized.

The core result is a parameterized unique segment decomposition theorem, obtained via a leftmost admissible termination rule. The construction is deterministic, finite, and recursively applicable.

1 Introduction

Chanlun theory describes price structures using hierarchical geometric objects such as fractals, strokes, and segments. However, existing formulations rely on informal descriptions and implicit conventions.

This paper constructs a formal system in which:

- All objects are defined over finite interval sequences;
- All rules are algorithmically specified;
- All parameters are explicit;
- Segment decomposition is uniquely determined.

2 Interval Sequences and Normalization

Definition 1 (Interval Sequence)**.** *A finite interval sequence is*

$$\mathcal{X} = (X_1, \ldots, X_n), \quad X_i = [L_i, H_i], \quad L_i \le H_i.$$

Definition 2 (Normalization with Provenance)**.**

$$\mathcal{N}(\mathcal{X}) = (\widehat{\mathcal{X}}, \pi)$$

where:

- $\widehat{\mathcal{X}}$ *has no adjacent containment;*
- $\pi(i)$ *records indices merged into* X_i*;*
- π *forms a partition and preserves order.*

3 Fractals

Definition 3 (Fractal)**.** *Let* $\widehat{\mathcal{X}} = (X_1, \ldots, X_u)$*.*

Top fractal:

$$H_i > H_{i-1},\ H_i > H_{i+1},\ L_i > L_{i-1},\ L_i > L_{i+1}$$

Bottom fractal:

$$L_i < L_{i-1},\ L_i < L_{i+1},\ H_i < H_{i-1},\ H_i < H_{i+1}$$

Lemma 1 (Fractal Completeness)**.** *Every interior position satisfies exactly one of: top fractal, bottom fractal, or neither.*

4 Stroke Construction

Definition 4 (Stroke Sequence)**.** *Fix* $\delta_{\min} \in \mathbb{N}_{>0}$*.*

Construct strokes by:

- *extracting fractals;*
- *selecting extremal representatives;*
- *connecting alternating fractals with separation* $\geq \delta_{\min}$*.*

Lemma 2 (Stroke Uniqueness)**.** *Given* $\delta_{\min}$ *and normalized input, the stroke sequence is unique.*

5 Feature Sequences

Definition 5 (Prefix Stroke Chain)**.**

$$\mathcal{B}[a, j] = (B_a, \ldots, B_j)$$

Definition 6 (Feature Sequence)**.**

$$\Phi(\mathcal{B}[a, j]) = \textit{strokes opposite to } B_a$$

Definition 7 (Normalized Feature Sequence)**.**

$$\widehat{\Phi}(a, j) = \mathcal{N}(\Phi(\mathcal{B}[a, j]))$$

Lemma 3 (Normalization Invariance)**.** *Feature sequence normalization preserves interval structure and supports fractal detection.*

6 Termination Structure

Definition 8 (First Feature-Fractal Index)**.**

$$\mathcal{J}_F(a) = \{t : \widehat{\Phi}(a, t) \textit{ has fractal}\}$$

$$j_0(a) = \min \mathcal{J}_F(a)$$

Definition 9 (r Mapping)**.**

$$r(a) = \max \pi(k)$$

Definition 10 (Gap / Overlap)**.**

$$X_{k-1} \cap X_k = \varnothing \quad (gap)$$

$$X_{k-1} \cap X_k \neq \varnothing \quad (overlap)$$

Definition 11 (Reverse Event)**.**

$$j_1(r) = \min\{t : \widehat{\Phi}^{rev}(r,t) \text{ } has\ fractal\}$$

Definition 12 (Convention Bundle)**.**

$$\Theta = (\delta_{\min}, \mathcal{N}, \rho)$$

Definition 13 (Termination)**.**

$$\mathrm{Term}(a,j) = 1$$

iff:

First class:

$$j = j_0(a),\ overlap$$

Second class:

$$j = j_1(r(a)),\ gap$$

7 Segment Construction

Definition 14 (Termination Set)**.**

$$\mathcal{J}(a) = \{j : \mathrm{Term}(a,j) = 1\}$$

Definition 15 (Leftmost Termination)**.**

$$j^*(a) = \min \mathcal{J}(a)$$

Definition 16 (Segment)**.**

$$\Sigma(a) = \begin{cases} \mathcal{B}[a,s], & \mathcal{J}(a) = \varnothing \\ \mathcal{B}[a, j^*(a)], & otherwise \end{cases}$$

8 Main Result

Theorem 1 (Parameterized Unique Segment Decomposition)**.** *Fix* $\Theta = (\delta_{\min}, \mathcal{N}, \rho)$*.*
Then every finite normalized interval sequence admits a unique decomposition:

$$\mathcal{B}_\Theta(\bar{\mathcal{P}}) = \Sigma(a_1) \sqcup \cdots \sqcup \Sigma(a_q)$$

with

$$a_{k+1} = j^*(a_k) + 1.$$

Proof. Termination: $a_{k+1} > a_k$ and bounded.
Existence: constructive.
Uniqueness: each segment endpoint is the minimum of a finite set. □

9 Discussion

This system achieves:

- full operationalization;
- explicit parameterization;
- deterministic decomposition;
- recursive applicability.

Remaining open directions include:

- parameter selection;
- higher-level recursion consistency;
- formal verification.

10 Conclusion

We have formalized Chanlun geometric decomposition into a fully specified system with a provably unique segment decomposition rule. This provides a rigorous foundation for further mathematical and computational exploration.

Acknowledgments

This manuscript was produced through an extended iterative collaboration with Claude (Anthropic), specifically the Claude Opus 4.6 model, during sessions in April 2026.

Claude's substantive contributions to the final system include: the run-length-encoding (RLE) reformulation of strokes as direction-labeling runs; the refinement-type treatment of normalized bar sequences with adjacent strict monotonicity; the endofunctor framing for recursive multi-level structure together with a contraction-based termination argument; the reformulation of overlap from a geometric theorem into a construction convention; and structured peer review across six iteration rounds that brought the segment decomposition layer from informal description to the fully operational formal system presented in Sections 5–8.

The iteration methodology itself—a structured critique loop between a human operator and an AI model acting as a compression engine, in which each round of critique was phrased as numbered, specific, actionable fixes and each response addressed them faithfully—is itself a contribution worth noting. It is the mechanism by which this manuscript reached publication-ready rigor in a compressed timeframe, and it may serve as a reference point for similar formalization efforts in other informal intellectual systems.

The author retains full intellectual responsibility for the content and conclusions of this manuscript. AI contributions are acknowledged here rather than credited as co-authorship, in accordance with current scholarly norms regarding AI-assisted work and the present legal standing of AI-generated contributions. Any errors are the author's alone.

A Appendix A: Normalization Algorithm

A.1 A.1 Algorithm Definition

We define the normalization operator $\mathcal{N}$ on a finite interval sequence

$$\mathcal{X} = (X_1, \ldots, X_n)$$

as a deterministic left-to-right procedure.

Algorithm $\mathcal{N}$:

1. Initialize an empty list $\widehat{\mathcal{X}}$.
2. For each interval X_i (from $i = 1$ to n):
   (a) Append X_i to $\widehat{\mathcal{X}}$.
   (b) While the last two elements A, B of $\widehat{\mathcal{X}}$ satisfy containment:

$$A \subseteq B \quad \text{or} \quad B \subseteq A$$

   i. Replace (A, B) by a merged interval C defined by the direction rule:

$$C = \begin{cases} [\max(L_A, L_B), \max(H_A, H_B)] & \text{(upward rule)} \\ [\min(L_A, L_B), \min(H_A, H_B)] & \text{(downward rule)} \end{cases}$$

   ii. Record provenance:

$$\pi(C) = \pi(A) \cup \pi(B)$$

A.2 A.2 Determinism

The algorithm is deterministic because:

- The scan order is fixed (left-to-right);
- The containment check is well-defined;
- The direction rule is fixed as part of $\mathcal{N}$.

Thus $\mathcal{N}(\mathcal{X})$ is uniquely determined.

A.3 A.3 Provenance Tracking

Each output interval X'_k carries a provenance set:

$$\pi(k) \subseteq \{1, \ldots, n\}$$

These sets form a partition of the input indices and preserve order:

$$i < j \Rightarrow \max \pi(i) < \min \pi(j)$$

—

B Appendix B: Segment Construction Algorithm

B.1 B.1 High-Level Procedure

Given a normalized sequence $\bar{\mathcal{P}}$:

1. Construct stroke sequence $\mathcal{B}_\Theta(\bar{\mathcal{P}})$.
2. Initialize $a_1 = 1$.
3. For $k = 1, 2, \ldots$:
   (a) Compute $\mathcal{J}(a_k)$.
   (b) If empty:
   - Output final segment $\mathcal{B}[a_k, s]$ and terminate.

   (c) Else:
   - Compute $j^*(a_k) = \min \mathcal{J}(a_k)$
   - Output $\Sigma(a_k) = \mathcal{B}[a_k, j^*(a_k)]$
   - Set $a_{k+1} = j^*(a_k) + 1$

—

B.2 B.2 Predicate Evaluation

To evaluate $\mathrm{Term}(a, j)$:

1. Compute $\widehat{\Phi}(a, j)$ via full recomputation.
2. Detect fractals.
3. If $j = j_0(a)$:
   - check gap/overlap
   - return first-class termination if overlap
4. Else:
   - compute reverse sequence
   - compute $j_1(r)$
   - return second-class termination if matched

—

C Appendix C: Complexity Analysis

C.1 C.1 Normalization

Each interval may trigger merging with previous intervals.

Worst-case complexity:

$$O(n^2)$$

Typical case:

$$O(n)$$

—

C.2 C.2 Feature Sequence Recomputations

For each (a, j):

- feature extraction: $O(n)$
- normalization: $O(n)$
- fractal detection: $O(n)$

Total per (a, j):

$$O(n)$$

—

C.3 C.3 Overall Complexity

Worst case:

$$O(n^3)$$

Reason:

- $O(n)$ starting points
- $O(n)$ candidates per start
- $O(n)$ per predicate evaluation

—

C.4 C.4 Optimization Note

Using incremental maintenance:

$$O(n^2)$$

However, this paper adopts full recomputation to ensure:

- determinism
- stateless evaluation
- offline/online equivalence

—

D Appendix D: Determinism and Reproducibility

D.1 D.1 Determinism

All components are deterministic:

- normalization $\mathcal{N}$
- stroke construction
- feature extraction
- termination predicates

Thus:

$$\text{SegDecomp}_\Theta(\bar{\mathcal{P}})$$

is a deterministic function.

—

D.2 D.2 Reproducibility Guarantee

Because:

- no hidden parameters
- no stateful updates
- full recomputation at each step

the system satisfies:

$$\text{offline result} = \text{online result}$$

—

D.3 D.3 Implementation Independence

Any implementation that:

- respects Θ
- implements $\mathcal{N}$ faithfully
- uses exact arithmetic comparisons

will produce identical outputs.

—

E Appendix E: Summary

This appendix provides:

- explicit algorithms
- complexity bounds
- determinism guarantees

Thus the formal system is:

- mathematically well-defined
- algorithmically implementable
- reproducible across environments